

Homework 1: POS Tagging

CS 577: Natural Language Processing

You must write your answers in the provided answer template. Failure to use the template will result in a score of 0 for the written portion of this homework. Use of Generative AI (AI) on writing the written portion of the homework is prohibited, and you must disclose all AI usage.

1 Overview

In this assignment, you will implement a part-of-speech (POS) tagger using PyTorch. Your task involves constructing word representations, implementing linguistic features, and training a Multi-Layer Perceptron (MLP) to predict the grammatical category of each word in a sentence.

You are provided with a boilerplate file `hw1.py`. Complete all blocks marked with `TODO` and preserve the function signatures and required class attributes. You will evaluate five specified configurations and submit a **typed** report using `answer_template.tex`. Keep supplied Python files unchanged except `hw1.py`. Do not hardcode predictions or use evaluation labels in prediction. Disclose any AI assistance in your report. The assignment is worth **100 points**: programming (45 points), conceptual questions (30 points), and experimental analysis (25 points).

2 Dataset and Provided Resources

For this homework, we will use an English part-of-speech dataset based on Universal Dependencies English GUM. Use only the distributed splits. Each line of `data/train.jsonl` and `data/valid.jsonl` is a JSON object containing:

- `"id"`: A unique sentence identifier (string).
- `"tokens"`: The original words of a sentence (list of strings).
- `"tags"`: One POS label per original word (list of strings).

The original word boundaries are fixed. Do not merge, split, or remove words when constructing the target labels. The 17 labels, in their required index order, are:

ADJ, ADP, ADV, AUX, CCONJ, DET, INTJ, NOUN, NUM,
PART, PRON, PROPN, PUNCT, SCONJ, SYM, VERB, X.

Use `train.jsonl` to fit vocabulary statistics and train model parameters. Use `valid.jsonl` to select checkpoints and complete the report. A separate test split is reserved for evaluation.

Use `prepare_resources.py` to download the resources linked in the coding README. It prepares the GUM training and validation splits, extracts Qwen3.8-27B's input embedding table, and uses your `extract_bert_words` function to cache BERT contextual word representations. You do not need to download or run the complete Qwen model.

3 Allowed Packages

Use Python 3.12 and the package versions specified in `requirements.txt`: `numpy`, `torch`, `tokenizers`, `transformers`, `safetensors`, and `matplotlib`, together with their dependencies

and the Python standard library. You may use PyTorch layers, automatic differentiation, and optimizers. Do not use an existing POS tagging pipeline, additional pretrained models, additional labeled data, or external prediction services.

4 Programming Section (45 pts)

Task 1: Data Loading and Word Context (8 pts)

Functions: `get_data`, `build_vocab`, `encode_words`, and `make_windows`.

Read the JSONL files and validate their records as specified in the docstrings. Ignore blank lines. Reject invalid labels, inconsistent token/label lengths, duplicate sentence IDs, and empty input files.

Build a word vocabulary using **only the training split**. Lowercase words with `str.lower()` and retain words occurring at least twice. Assign IDs in lexicographic order starting at 2. Reserve ID 0 for <PAD> and ID 1 for <UNK>. Map every excluded or unfamiliar word to <UNK> without changing the vocabulary.

For each word, construct an ordered context window. Configuration 1 uses radius 0; configurations 2, 3, and 4 use radius 2. A radius-2 window contains the two preceding words, the target word, and the two following words. Pad outside the sentence with zeros. Windows must not cross sentence boundaries. For example:

$$[2, 3, 4] \longrightarrow \begin{bmatrix} 0 & 0 & 2 & 3 & 4 \\ 0 & 2 & 3 & 4 & 0 \\ 2 & 3 & 4 & 0 & 0 \end{bmatrix}.$$

The window function must also accept a matrix of word vectors and preserve gradient flow through the input vectors.

Task 2: Linguistic Feature Engineering (7 pts)

Class: `LinguisticFeatures`.

Methods: `fit` and `transform`.

Construct the following feature vector for the **target word**:

1. Three separate one-hot suffix blocks for suffix lengths 1, 2, and 3. Lowercase words before extracting suffixes. For each length, retain suffixes occurring at least twice in training; assign IDs in lexicographic order after <UNK> at index 0. Words shorter than the suffix length, and suffixes absent from the fitted dictionary, use that block's <UNK> entry.
2. Five binary flags, in this order: the first character is uppercase; the word is uppercase according to `str.isupper()`; the word contains a digit according to `str.isdigit()`; the word contains the ASCII hyphen -; and the word is the first word of its sentence. Compute flags from the original text.

Concatenate the three suffix blocks followed by the five flags. Return a `float32` tensor. Do not fit or modify feature dictionaries using validation or test data.

Task 3: MLP and Word-Embedding Taggers (10 pts)

Classes: `POSMLP` and `WindowTagger`.

Implement the following classifier, using biases in both linear layers:

$$\text{Input} \rightarrow \text{Linear}(D_{\text{in}}, 128) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.1) \rightarrow \text{Linear}(128, 17).$$

Return raw logits; do not apply softmax before cross-entropy loss. Store these layers in `POSMLP.network`. The constructor must honor the dimensions supplied by its arguments, including smaller dimensions used in the public tests.

For configurations 1, 2, and 3, create a trainable word embedding table with 100 dimensions per word. Initialize it from $\mathcal{N}(0, 0.02^2)$ and set the padding row to zero. Use `padding_idx=0`. Store the table as `WindowTagger.embedding` and the classifier as `WindowTagger.classifier`.

Flatten window embeddings in left-to-right order. Configuration 3 appends Task 2's feature vector after the flattened embeddings. Configuration 1 has input dimension 100; 2 has input dimension 500; 3 has input dimension $500 + F$, where F is the feature-vector dimension.

Task 4: Pretrained Subword Representations (8 pts)

Functions: `tokenize_word_pieces` and `mean_pool_subwords`.

Class: `QwenTagger`.

For configuration 4, use the supplied Qwen tokenizer and extracted **input embedding table**. Tokenize each original word separately, without lowercasing, adding leading spaces, inserting special tokens, or applying a chat template. The required call is:

```
tokenizer.encode(word, add_special_tokens=False).ids
```

This convention enforces the dataset's original word boundaries. Do not replace it with tokenization of the joined sentence.

For a word represented by m subwords, form its word vector by averaging their input embeddings:

$$\mathbf{v}(w) = \frac{1}{m} \sum_{j=1}^m \mathbf{e}(s_j).$$

Ignore padding in the mean. Return a zero vector for an all-padding row in the pooling function. The supplied utilities perform embedding lookup and reuse the pooled vectors; the embedding table remains **frozen**.

Construct a five-word context window from the pooled word vectors: the target word, its two preceding words, and its two following words. Concatenate the vectors in left-to-right order and pass them to the Task 3 MLP. If the Qwen embedding dimension is d , the MLP input dimension is $5d$. Keep the input embeddings frozen and train only the MLP.

A projection is optional. To reduce the input dimension, you may apply a shared trainable linear layer to each word vector before concatenation. Use no bias or activation so zero boundary padding stays zero. Set `projection_dim` to `null` in the configuration file for direct input (the default), or to a positive integer p for a projection; the MLP then receives $5p$ features. Store `nn.Identity()` or the linear layer as `QwenTagger.projection`, and the MLP as `QwenTagger.classifier`. If enabled, train the projection together with the MLP. Use the same choice for both seeds and report it with the parameter count.

Task 5: Contextual Word Representations (6 pts)

Functions: `first_subword_indices` and `extract_bert_words`.

Configuration 5 uses the final hidden layer of frozen **bert-base-cased**. Tokenize the **complete list of original words** with a fast tokenizer using `is_split_into_words=True`. Include the model's special tokens, retain the attention mask, and do not silently truncate inputs.

Use the first subword position corresponding to each original word to select one contextual vector. Exclude special tokens and padding from the selected vectors. Reject missing or invalid word alignments. If the encoded sentence exceeds the encoder's maximum length, raise `ValueError`. Set the encoder to evaluation mode, disable gradients for its parameters, and extract vectors under `torch.no_grad()`. Return detached `float32` CPU vectors in original word order.

After implementing `extract_bert_words`, run `prepare_resources.py` to cache BERT contextual word representations for the full experiments. Because BERT is frozen, reuse these cached representations across epochs and seeds. Rerun preparation if you change

`extract_bert_words`. The provided `BertTagger` applies the Task 3 MLP to each contextual word vector; do not add a five-word window. Train only the MLP.

Task 6: Training and Evaluation (6 pts)

Functions: `train_epoch` and `evaluate`.

Implement one training epoch with cross-entropy loss and the optimizer passed to the function. Set training mode, clear gradients before each update, run the forward pass, compute the loss, backpropagate, and update the trainable parameters. Do not recreate the optimizer between epochs.

For evaluation, set evaluation mode, disable gradient computation, and return predictions in input order. Exclude labels equal to -100 from both loss and accuracy, and skip all-ignored training batches. Raise `ValueError` if an entire epoch or evaluation call contains no valid labels. Report loss as the **mean over valid words**, rather than an unweighted mean of batch losses. Accuracy must be a fraction between 0 and 1. Evaluation must not update model parameters.

5 Required Experiments

Complete **all five** configurations below using the distributed `config.json`. Use the representations below; the Qwen projection described in Task 4 is optional.

Model	Representation	Trainable components
1	Word embedding of the target word	Embeddings and MLP
2	Word embeddings in a five-word context window	Embeddings and MLP
3	Word embeddings in a five-word context window, plus linguistic features of the target word	Embeddings and MLP
4	Qwen subword embeddings in a five-word context window	MLP; optional projection
5	BERT contextual representation of the target word	MLP

For model 4, average the subword embeddings within each word before constructing the window. For model 5, use the target word’s contextual vector from the full sentence.

Use seeds **2026** and **577**, 10 epochs, batch size 256, Adam with learning rate 10^{-3} and zero weight decay, hidden width 128, and dropout 0.1. Shuffle training examples each epoch; do not shuffle evaluation examples. The supplied runner applies these settings and records results. Select the checkpoint with the highest overall validation accuracy; on a tie, retain the earliest epoch. Do not select checkpoints using subset scores. After preparing the resources, run these commands from `code/`:

```
python run_experiments.py --config config.json
python summarize_results.py --results results
```

You may run a subset of configurations while debugging with `--models 1 2 3`. Report results for all five configurations and both seeds. Your submitted code must reproduce all reported results using the supplied resources and specified settings. Record any optional projection setting in your report. Public tests use small synthetic inputs and run without network access or pretrained checkpoints.

6 Conceptual Questions (30 pts)

Answer each question in the corresponding section of the answer template. Use a few short sentences unless a calculation or table is requested. Show intermediate steps for calculations.

Q1: Grammatical Ambiguity (5 pts)

Identify the POS tag of *can* in “They can swim” and “The can leaked.” Explain why a deterministic model that receives only the same word embedding cannot always distinguish these uses. In a toy dataset, *book* appears six times as a noun and four times as a verb. What is the highest accuracy a deterministic word-identity-only classifier can achieve on those ten occurrences? What additional information could change this limit?

Q2: Morphology and Orthography (5 pts)

Explain why suffix features can help tag words that were not observed during supervised training. Give a counterexample to the rule “every word ending in *-ly* is an adverb.” Explain why capitalization can help identify proper nouns and why the sentence-initial flag is relevant to interpreting that feature.

Q3: Subword Tokenization (5 pts)

Compare word, subword, and character tokenization in terms of vocabulary size, sequence length, and the representation of unfamiliar words. Using the supplied Qwen tokenizer and Task 4’s convention, record the actual pieces for *walk*, *walked*, *walking*, *unhappiness*, and *microservices*. Do the pieces necessarily correspond to morphemes? What information does mean pooling discard? Explain why tokenizing isolated words can differ from tokenizing a complete sentence.

Q4: Dimensions and Parameters (5 pts)

Let $V = 10,000$ include the two reserved vocabulary entries. Use embedding dimension 100, hidden width 128, and 17 output labels. Calculate the MLP input dimension, embedding parameter count, MLP parameter count, and total parameter count for models 1 and 2. Count the padding row as an embedding parameter even though its gradient is suppressed. Include both linear-layer biases. Explain what information is lost if the five word embeddings in model 2 are averaged instead of concatenated.

Q5: Supervision and Gradient Flow (5 pts)

Consider the BERT alignment

[None, 0, 0, 1, 2, 2, None, None].

Give the zero-based subword positions selected for the three original words. Explain why scoring every subword as a separate prediction changes the weight assigned to different words. Identify the trainable components in models 4 and 5.

Q6: Interpreting Comparisons (5 pts)

Suppose model 5 achieves higher validation accuracy than model 4. Does this establish that contextualization alone caused the difference? Identify at least three factors that differ between these configurations. Explain why model 4’s input embedding lookup is static even though the vectors were learned as part of an LLM. Finally, explain why validation results must not be reported as untouched test-set results.

7 Experimental Analysis (25 pts)

Use the blank tables and figure placeholders in `answer_template.tex`. Report validation results from the selected checkpoints. Do not invent missing values.

1. **Results and reproducibility (8 pts).** Report the mean and population standard deviation across the two seeds for overall accuracy, non-punctuation accuracy, unseen-word accuracy, and ambiguous-word accuracy. Express accuracies as percentages and deviations in percentage points. Include the number of evaluated words in each subset, trainable parameter counts, and the selected epoch for each seed. For an empty subset, report N/A and a count of 0. Use $\sigma = \sqrt{\frac{1}{2} \sum_{i=1}^2 (a_i - \bar{a})^2}$.
2. **Learning curves (4 pts).** For each of the five models, plot training loss and validation loss together over the 10 epochs using seed 2026. Label both curves and the axes. Use cross-entropy loss and include all 10 epochs, even if an earlier checkpoint was selected. The training loss is measured during updates; the validation loss is measured after each epoch.
3. **Representation comparisons (8 pts).** Answer four short questions: What changes from 1 to 2? What changes from 2 to 3? How does 4 compare with 2 on unseen words? How does 5 compare with 4 on ambiguous words? Cite numerical results and identify limitations of each comparison. In addition, report model 4's accuracy and example counts for words represented by one, two, and three-or-more Qwen subwords. Explain whether differences among these groups establish a causal effect of segmentation.
4. **Error analysis (5 pts).** Use the errors list in `results/2_seed2026.json`: the first five incorrect validation predictions from model 2's selected checkpoint, sorted by sentence ID and zero-based word index. This list depends on your model. For each, record the sentence, target word, gold tag, predicted tag, and a plausible explanation grounded in the available features. If fewer than five errors exist, analyze all of them and mark unused rows N/A.

Lowercase words using the same rule as Task 1 when determining unseen and ambiguous status from the training data. For Qwen subword-count groups, tokenize each original-case word using the same convention as configuration 4. Determine non-punctuation status from the gold tag. An **unseen word** has zero occurrences in the original training split, even if a pretrained tokenizer can represent it. An **ambiguous word** occurs with at least two distinct gold tags in training. A **non-punctuation word** has a gold tag other than PUNCT. These definitions are fixed across configurations; excluded low-frequency vocabulary entries are not automatically unseen words.

8 Submission

On `data.cs.purdue.edu`, install the provided testing wheel following the coding README. From your code directory, run:

```
hw1 check --submission hw1.py
hw1 submit --submission hw1.py
```

The checker reports points for each check and a total coding score out of 45. Submit only the compiled report PDF to **Gradescope**, using the provided answer template. Do not submit LaTeX source, results, checkpoints, or datasets.